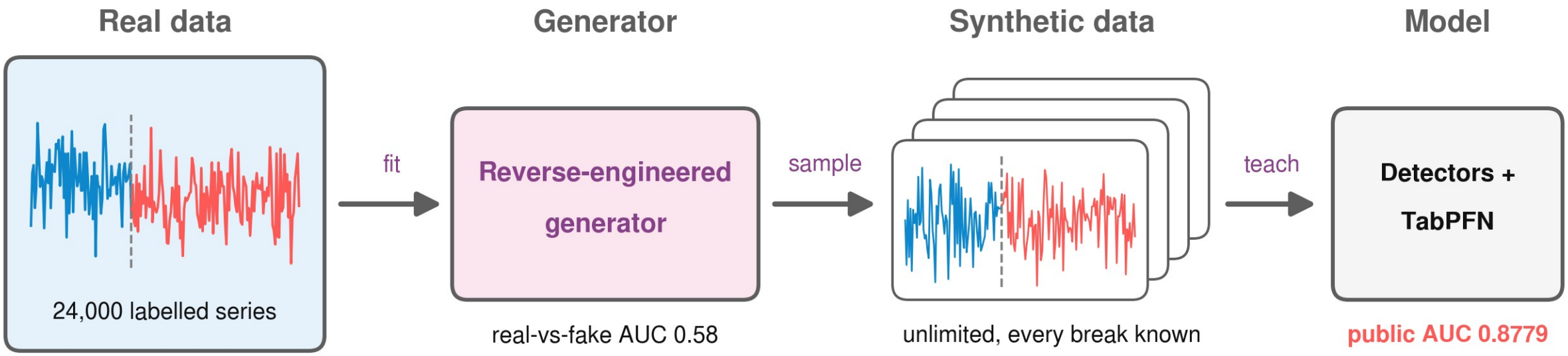
Let the generator teach the model

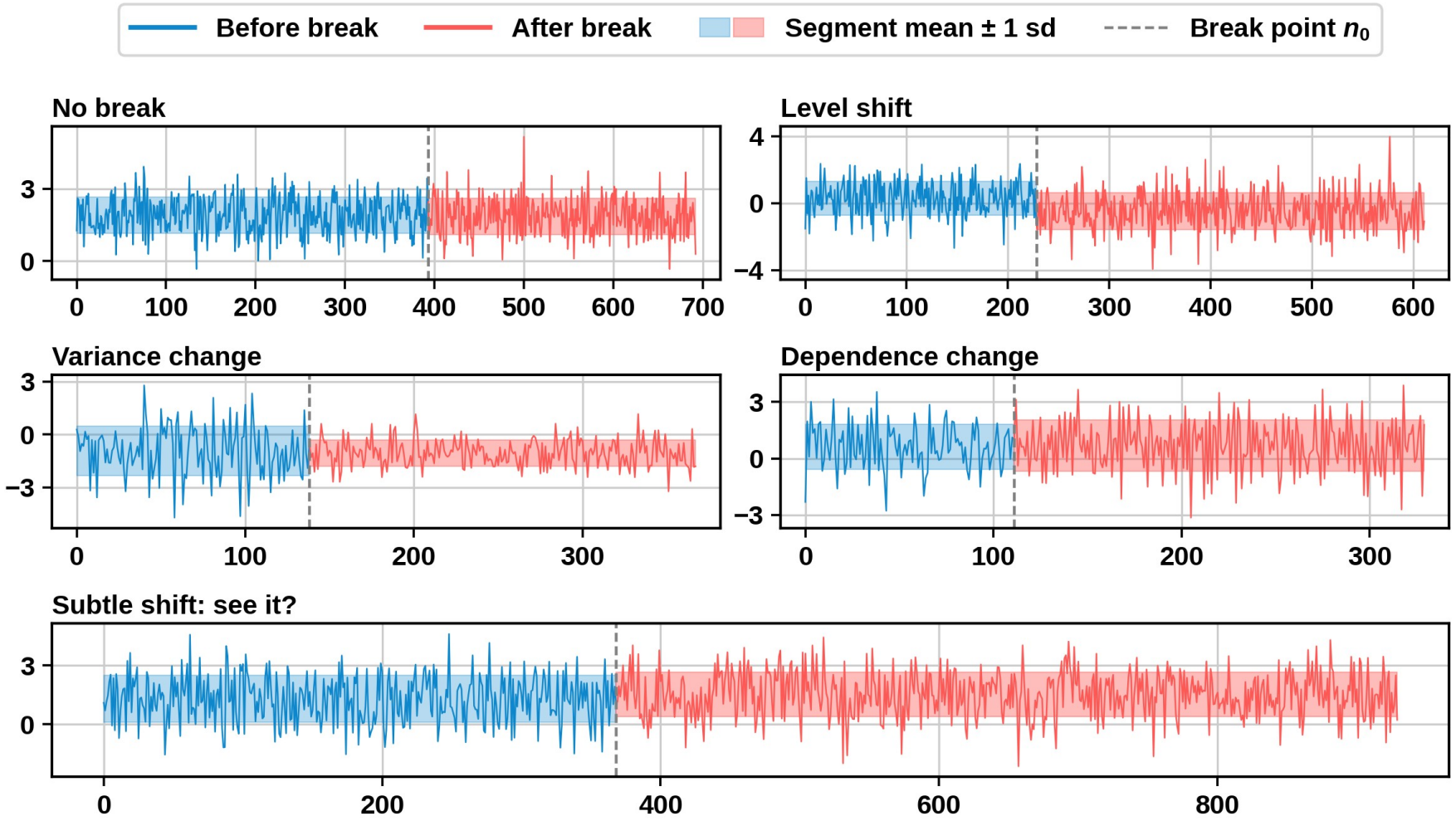
Structural-break detection: rebuild how the data was made, then learn from synthetic data



0.8779 public AUC **0.8741** out-of-fold AUC **#2** of 6 finalists after qualification

Team BreakPoint • KU Applied AI Challenge 2026 (KU-SBB) • Final, 11 November

Breaks range from obvious to invisible



Synthetic examples from our generator. Dashed line = known boundary n0; band = segment mean ± 1 sd.

The task

Known boundary: did the process change there? ROC-AUC on 4,000 test series.

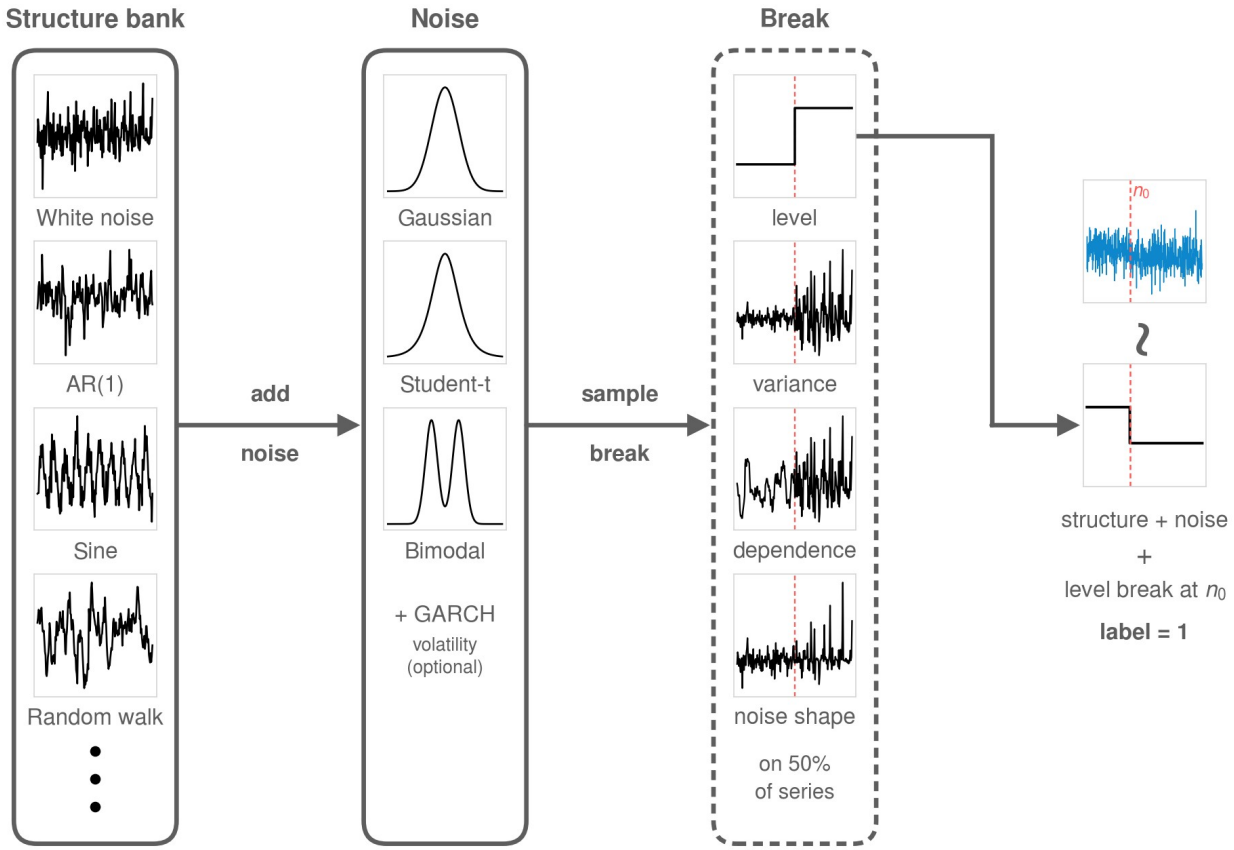
What can change

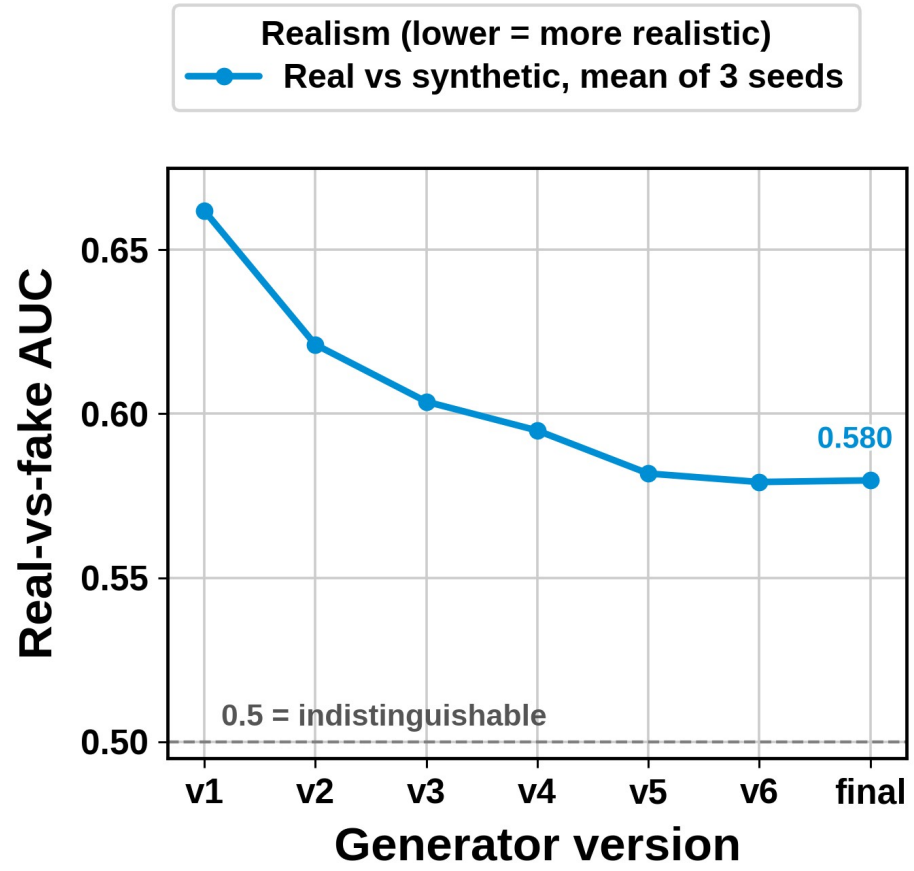
Level, variance, dependence or the noise distribution.

Why it is hard

About 1 in 3 level / variance breaks is as small as the last panel.

We reverse-engineered the generator behind the data



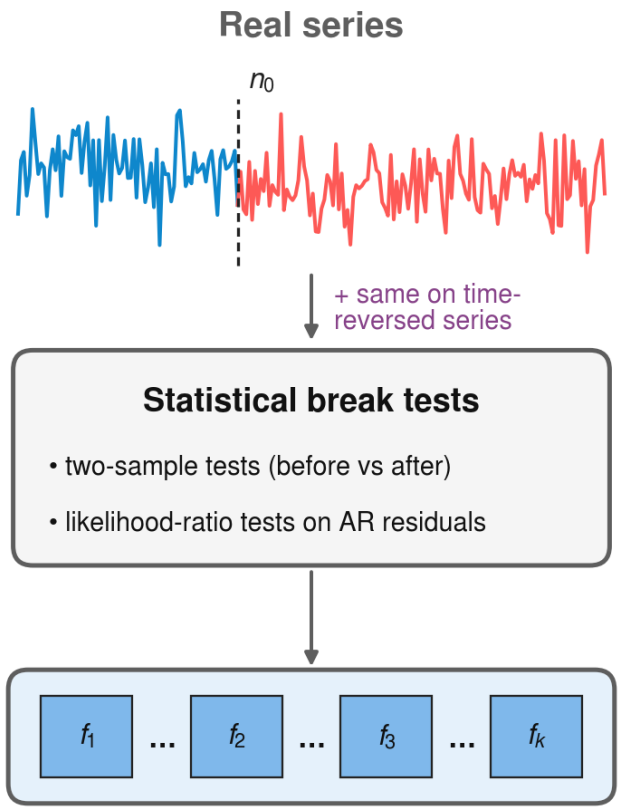


Recipe fitted by matching real series (ku_gen.py in the repo): structure → noise → optional break at n0

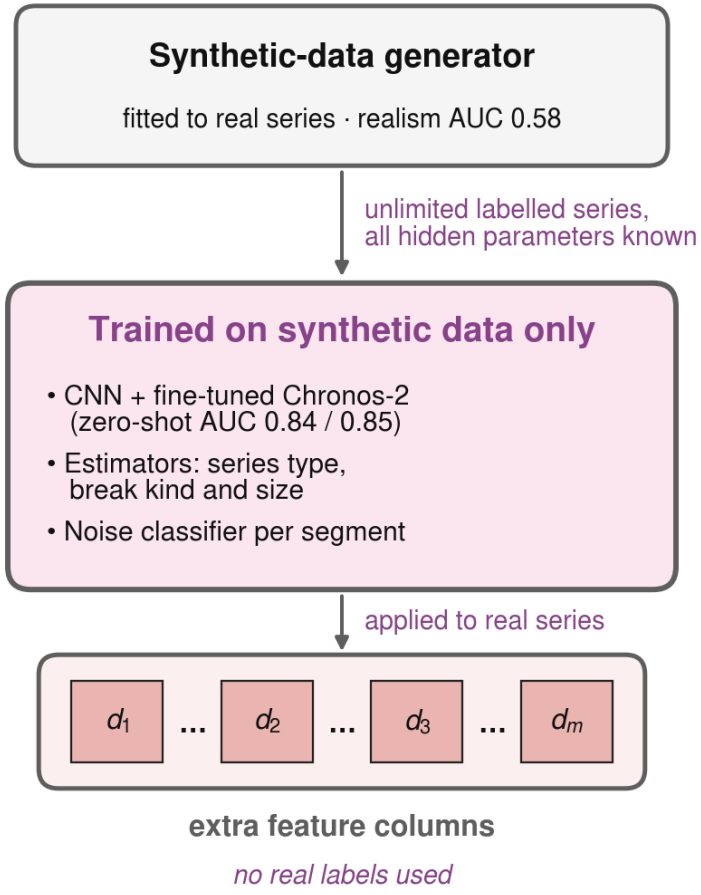
Mean of 3 seeds, noise ±0.003

Label-free signals feed a TabPFN ensemble

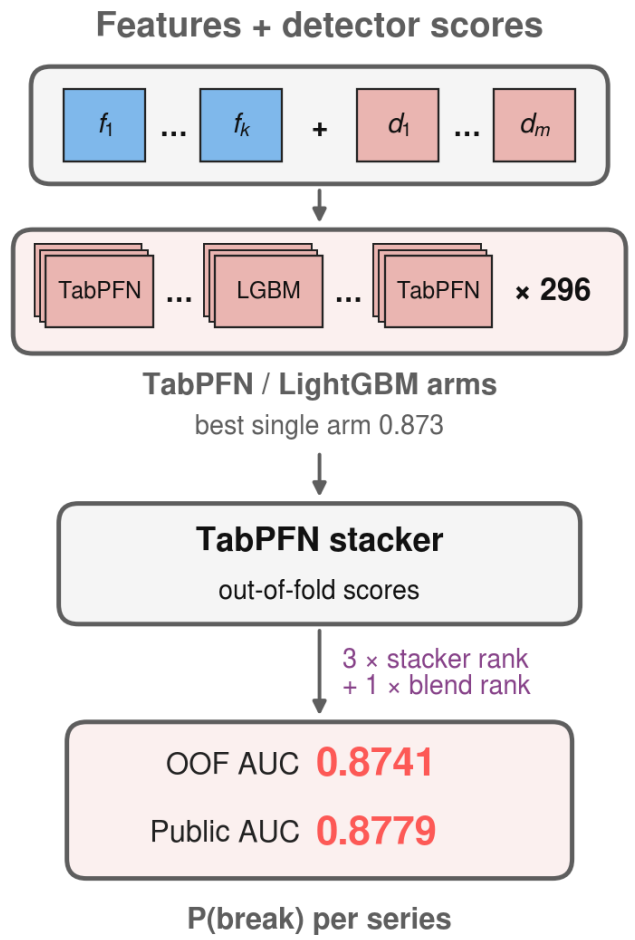
Real series → features



Generator → label-free detectors



Ensemble → prediction



Every gain had to survive fold-clean validation

1

Fold-clean

5-fold stratified CV; selection, TabPFN context and stacker fitted inside each training fold

2

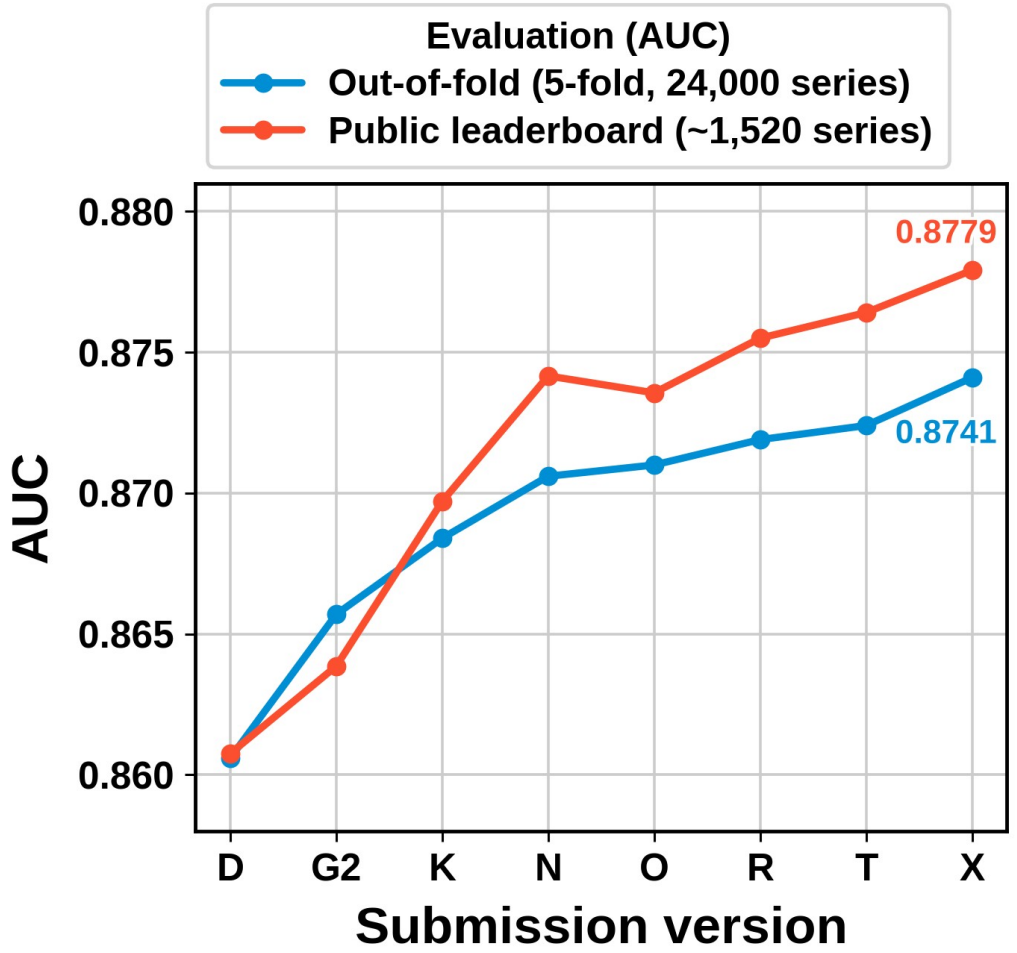
Two-stage test

Fast screen, then full-setting confirm with a paired bootstrap CI; quick gains usually halve

3

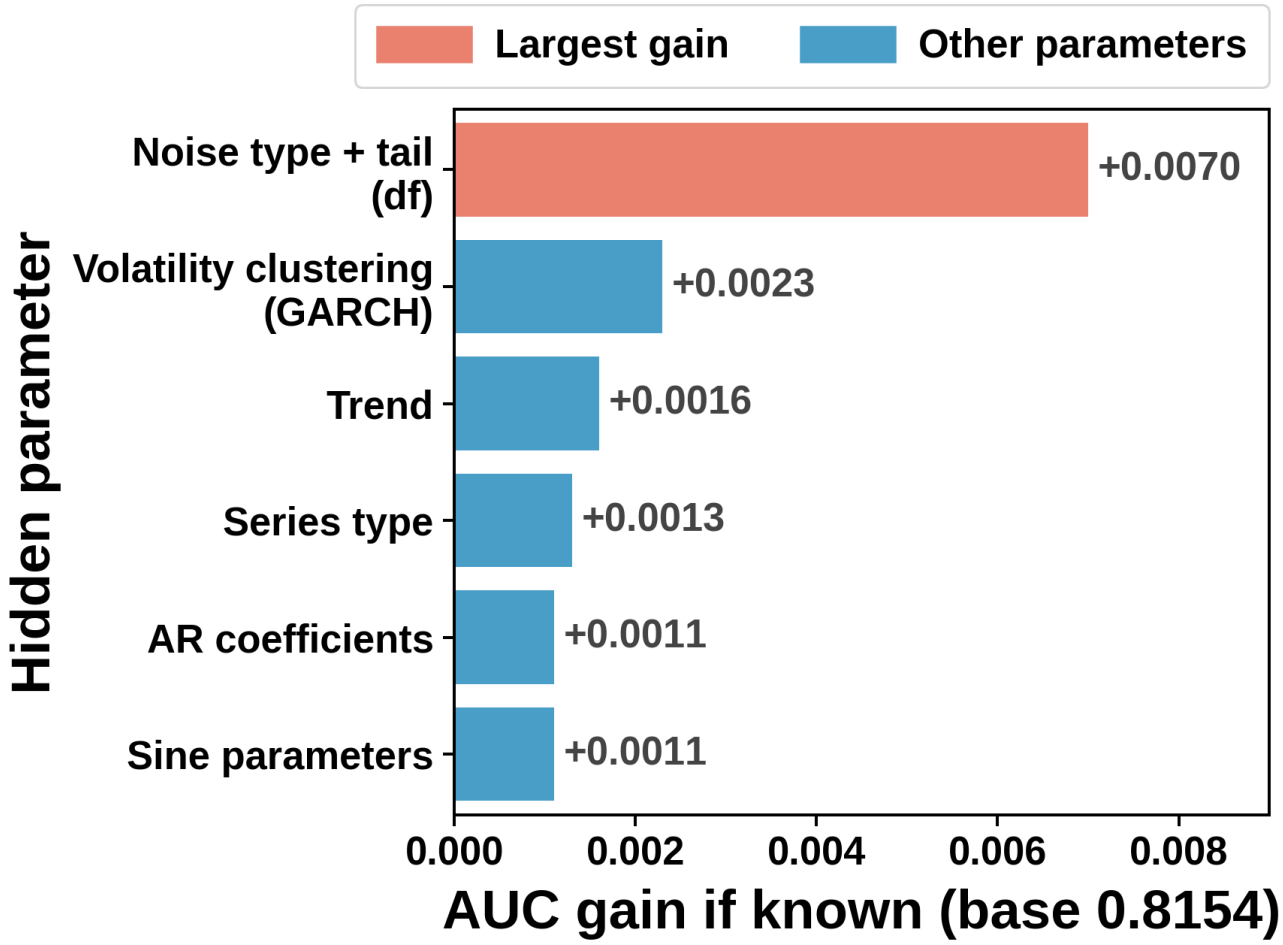
Decide by OOF

Public LB = ~1,520 series, noise ±0.009 (larger than most gains); OOF on 24,000 series is ±0.002



D = CPU-auditable LightGBM, G2 = qualification, X = best (blendXFS)

An oracle experiment showed where the fixable error is



LightGBM on 90k synthetic series, tested on 30k held out; baseline 0.815

How the oracle works

1. **Synthetic data**
   every hidden parameter of every series is known

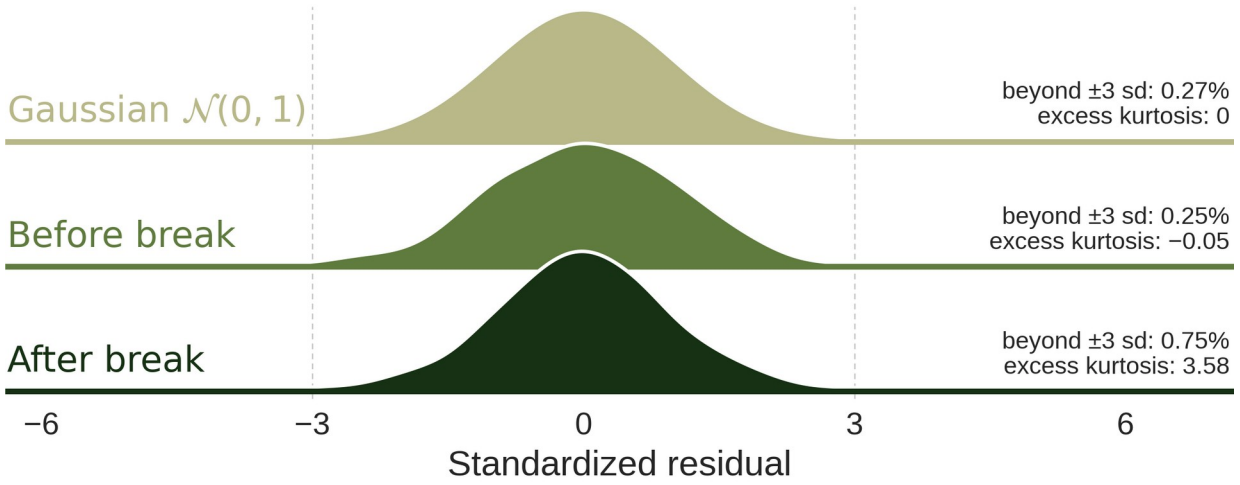
2. **Tell the model**
   add the true value as an extra feature, retrain

3. **Measure**
   AUC gain = what better estimation of it is worth

Biggest lever: the noise distribution before the boundary

A noise classifier per segment turned this into score

Same spread, heavier tails after the break



One synthetic series; AR(5) residuals, each segment standardized to sd 1

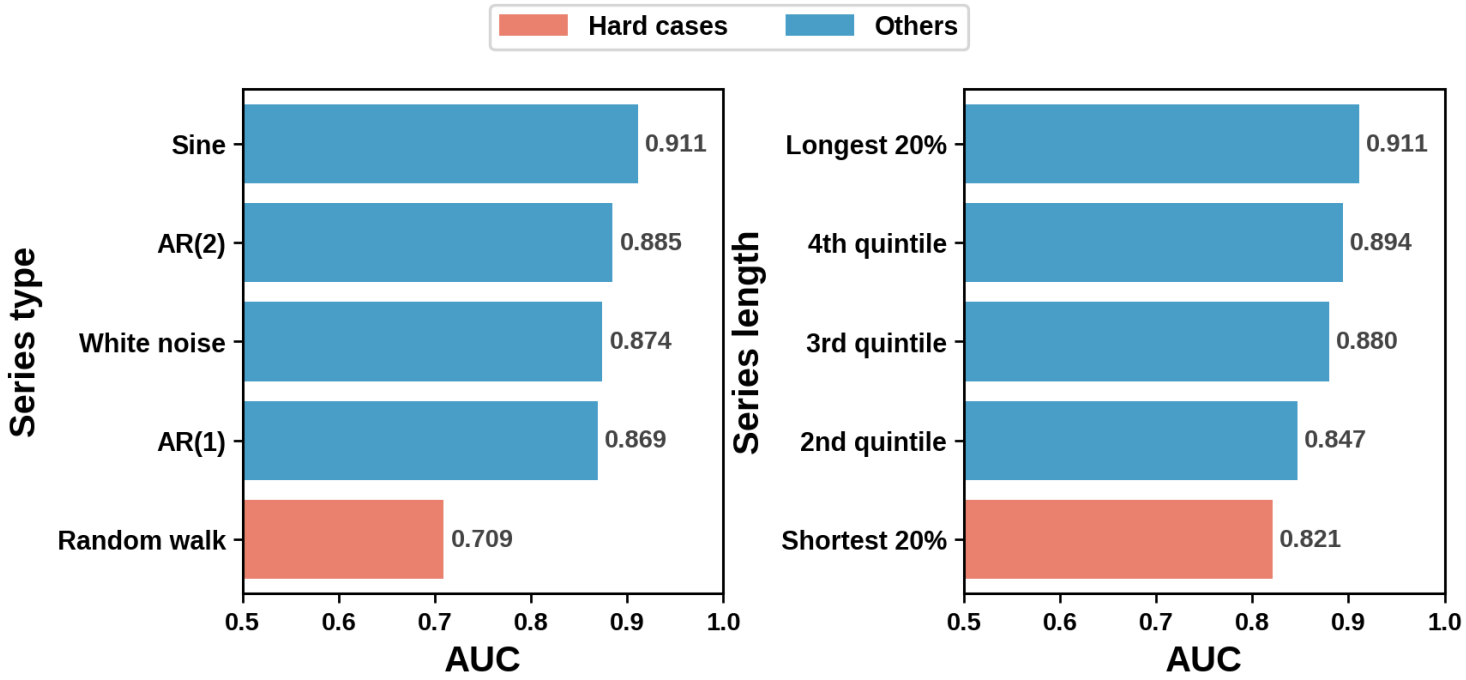
Same pipeline, plus the noise classifier

Best LightGBM	0.8567 → **0.8588**
Best TabPFN arm	0.8716 → **0.8729**
Stacker (OOF)	0.8733 → **0.8741**
Public AUC	0.8764 → **0.8779**

LightGBM gain confirmed: +0.0021, 95% CI +0.0011 to +0.0032

Limitations: hard series, compute, generator

Out-of-fold AUC: errors concentrate in random walks and short series



Hard series

Random walks (0.71) are equally hard on synthetic data, so this looks intrinsic; short series are harder too.

Compute

Best model needs a GPU and 296 arms; the best single arm is within 0.0012, so most arms are redundant.

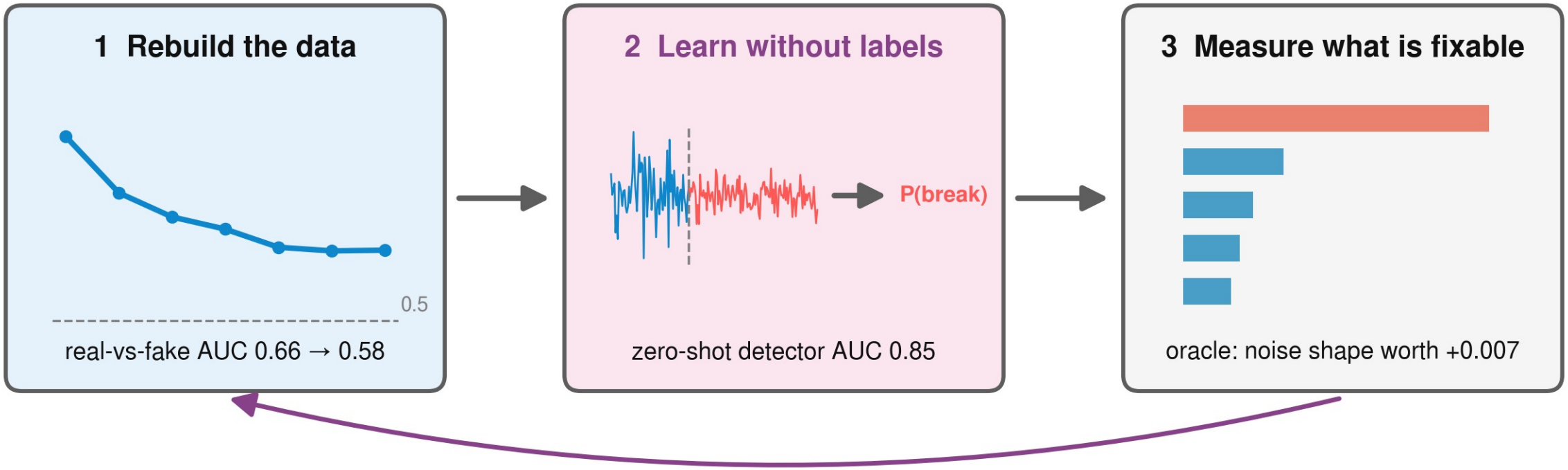
Audit route

A bit-exact LightGBM route (0.861 public) runs in 31 s on 2 CPU cores; the GPU route is documented step by step.

Generator

Still detectable (0.58); synthetic breaks are slightly smaller than real ones.

Model how the data was made, then learn from it



the measured gap tells us what to fix next

Thank you — questions welcome

Code: github.com/zhuhxi/ku-sbb-breakpoint (tag final) • Fine-tuned Chronos-2 checkpoints (KU OneDrive, SHA-256 inside): link